

The Lain Programming Language

Language Specification

Version 0.1 — Draft

April 23, 2026

Document status: Draft — not for normative use
Language version: Lain 1.0
Date: April 23, 2026

This document is the authoritative specification of the Lain programming language. It defines the syntax, semantics, and constraints that every conforming implementation shall enforce.

Contents

Foreword	vii
Introduction	ix
1 Scope	1
1.1 Compilation model	2
2 Normative references	3
3 Terms, definitions, and symbols	5
3.1 General language terms	5
3.2 Type system terms	7
3.3 Ownership and memory terms	8
3.4 Control flow and purity terms	9
3.5 Verification terms	9
3.6 Module and name resolution terms	10
4 Conformance	11
4.1 Conforming implementation	11
4.2 Conforming program	11
4.3 Diagnostic messages	12
4.4 Implementation limits	12
4.5 Safe and unsafe fragments	13
5 Lexical conventions	15
5.1 Source file encoding	15
5.2 Comments	15
5.3 Tokens	15
5.4 Whitespace	16
5.5 Keywords	16
5.6 Identifiers	17
5.7 Literals	18
5.7.1 Integer literals	18
5.7.2 Floating-point literals	18
5.7.3 Boolean literals	18
5.7.4 Character literals	18
5.7.5 String literals	19
5.7.6 Escape sequences	19
5.8 Operators and punctuators	19
5.8.1 Arithmetic operators	20
5.8.2 Comparison operators	20
5.8.3 Logical operators	20
5.8.4 Bitwise operators	20

5.8.5	Assignment operators	20
5.8.6	Other operators and punctuators	21
5.9	Token disambiguation	21
5.10	Integer arithmetic and overflow	22
6	Basic concepts	23
6.1	Declarations and definitions	23
6.2	Scopes	23
6.3	Name resolution	24
6.4	Storage duration	24
6.5	Objects and values	24
6.6	Lvalues and rvalues	24
6.7	Alignment	25
7	Types	27
7.1	Type taxonomy	27
7.2	Primitive types	27
7.2.1	Integer types	27
7.2.2	Integer rank and widening	28
7.2.3	Floating-point types	29
7.2.4	Boolean type	29
7.2.5	The void type	29
7.3	Array types	29
7.4	Slice types	30
7.5	Struct types	30
7.6	Enum types	30
7.7	Algebraic data types (ADTs)	31
7.8	Pointer types	31
7.9	Constrained types	31
7.10	Type compatibility and equivalence	32
8	Expressions	33
8.1	Operator precedence and associativity	33
8.2	Primary expressions	33
8.3	Arithmetic operators	34
8.4	Comparison operators	34
8.5	Logical operators	34
8.6	Bitwise operators	34
8.7	The <code>in</code> operator	35
8.8	Cast expression (<code>as</code>)	35
8.9	Move expression (<code>mov</code>)	36
8.10	Borrow expressions (<code>ref</code> , <code>mut</code>)	36
8.11	Field access	36
8.12	Index expression	36
8.13	Function call	36
8.14	Block expression	37
8.15	If expression	37
8.16	Match expression (<code>case</code>)	37
8.17	Constructor expression	37
8.18	Compound assignment	37
9	Statements	39

9.1	Block statement	39
9.2	Variable declaration statement	39
9.3	Assignment statement	39
9.4	Expression statement	40
9.5	If statement	40
9.6	While statement	40
9.6.1	Bounded while statement	41
9.7	Return statement	41
9.8	Defer statement	42
9.9	Unsafe block	42
9.10	Comptime if statement	42
9.11	For statement	42
10	Declarations	45
11	Ownership	47
12	Functions	49
13	Vra	51
14	Generics	53
15	Match	55
16	Modules	57
17	Interop	59
18	Unsafe	61
19	Memory	63
20	Stdlib	65
	Grammar	67
	Diagnostics	69
	Rationale	71
	Comparison	73
	Index	75

Foreword

This document specifies the Lain programming language, Version 1.0.

Lain is a statically-typed, compiled programming language for systems programming. It guarantees memory safety, type safety, and resource safety entirely at compile time with zero runtime overhead.

This specification is structured analogously to ISO/IEC 9899 (the C standard). Clauses 1 through 4 are administrative; Clauses 5 through 20 are normative. Annexes A and B are normative; Annexes C and D are informative.

Text marked **Constraint** describes a condition whose violation requires an implementation to issue a diagnostic message. Text marked *implementation-defined*¹ denotes behavior that is valid but whose exact result is determined by the implementation and shall be documented by it. Text marked **undefined behavior**² denotes a situation for which this specification imposes no requirements.

Note

Undefined behavior in Lain can arise only inside **unsafe** blocks or through direct C interoperability. All code in the safe fragment of Lain has defined behavior.

¹Implementation-defined behavior:

²Undefined behavior:

Introduction

Design philosophy

Lain is built on five foundational pillars:

- P1 — Assembly-Speed Performance** Every language feature compiles to C code that is equivalent in performance to hand-written C. There are no hidden allocations, no garbage collector, and no reference-counting overhead.
- P2 — Zero-Cost Memory Safety** Memory safety is enforced entirely at compile time through a linear type system, a borrow checker, and value range analysis. A conforming implementation that rejects all constraint violations guarantees, for safe programs, the absence of use-after-free, double-free, null dereference, and buffer overflow at runtime.
- P3 — Static Verification** The verification properties required by this specification are decidable and polynomial in program size. No SMT solver or unbounded fixpoint iteration is required.
- P4 — Determinism** Pure functions (`func`) are referentially transparent and guaranteed to terminate. The same inputs always produce the same outputs.
- P5 — Syntactic Simplicity** Lain has no implicit copies, no hidden conversions (beyond the defined integer-widening rules), and no magic methods. Ownership is always explicit.

Relationship to C

Lain compiles to C99. The C code generated by a conforming Lain implementation is the *translation output*; this specification does not prescribe the form of that output beyond the observable behavior of the resulting program.

How to read this specification

Each normative clause is structured as follows: a prose description of the syntax (with a reference to the formal grammar in Annex A), followed by the semantics, followed by **Constraint** blocks listing every condition that requires a diagnostic message. Non-normative **Note** and **Example** environments appear throughout to aid understanding.

Chapter 1

Scope

- 1 This document specifies the Lain programming language. It defines the syntax, the semantic rules, and the constraints that every conforming Lain implementation shall enforce. It serves as the single authoritative reference for language users and implementors alike.
- 2 Lain is a statically-typed, compiled programming language designed for systems programming, embedded systems, and safety-critical software. It achieves memory safety, type safety, and resource safety entirely at compile time with zero runtime overhead.
- 3 This specification covers:
 - lexical structure and grammar (§5);
 - type system and type inference (§7);
 - variable declarations, scoping, and initialization (§6, §10);
 - expressions, operators, and evaluation order (§8);
 - statements and control flow (§9);
 - functions, procedures, and purity (§12);
 - ownership, borrowing, and the linear type system (§11);
 - value range constraints and static verification (§13);
 - compile-time evaluation and generics (§14);
 - pattern matching and exhaustiveness checking (§15);
 - module system (§16);
 - C interoperability (§17);
 - unsafe code boundaries (§18);
 - memory model (§19);
 - diagnostic messages (Annex B);
 - standard library (§20).
- 4 This specification does *not* cover:
 - the internal architecture of any particular compiler implementation;
 - build system integration or tooling;
 - IDE or editor support;
 - operating system interfaces beyond what the standard library provides.

1.1 Compilation model

- 1 Lain uses a source-to-C transpilation model. A Lain source file (extension `.ln`) is processed by a conforming implementation to produce C99 source code, which is subsequently compiled by a C99 compiler to produce an executable. The observable behavior of the program is defined by this specification; the form of the intermediate C99 code is *implementation-defined*¹.
- 2 A conforming implementation shall perform the following translation phases in the following order:
 1. **Lexical analysis** — source text is converted to a token stream as described in §5.
 2. **Parsing** — the token stream is reduced to an abstract syntax tree according to the grammar in Annex A.
 3. **Name resolution** — identifiers are resolved to declarations as described in §6.
 4. **Type checking** — types are inferred and verified as described in §7 through §10.
 5. **Exhaustiveness checking** — pattern matching completeness is verified as described in §15.
 6. **Use analysis (NLL)** — last-use points are computed for all variables.
 7. **Linearity and borrow checking** — ownership and borrowing rules are verified as described in §11.
 8. **Value range analysis** — bounds safety and numeric constraints are verified as described in §13.
 9. **Code emission** — well-formed programs are translated to C99.
- 3 A program that completes all phases without a diagnostic message is **well-formed** and its translation output is defined.

Note

Phases 3–8 collectively form *semantic analysis*. An implementation may interleave these phases as long as the observable behavior is equivalent to executing them in the order listed.

¹Implementation-defined behavior: the generated C99 code need not be human-readable

Chapter 2

Normative references

- ¹ The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies.

ISO/IEC 9899:2018 (C17)

Information technology — Programming languages — C.

International Organization for Standardization, Geneva, 2018.

Referenced for: C99/C17 calling conventions (§ 17), standard integer types and their representations (§ 7), and undefined behavior categories (§ 18).

IEC 60559:2020 (IEEE 754-2019)

Floating-point arithmetic.

International Electrotechnical Commission, Geneva, 2020.

Referenced for: semantics of `f32` and `f64` arithmetic (§ 7).

- ² The following documents are informative references. They are not required for conformance but provide useful background.

ISO/IEC 14882:2020 (C++20)

Programming languages — C++.

Referenced for: vocabulary and structure conventions used in this document.

The Rust Reference

The Rust Reference, <https://doc.rust-lang.org/reference/>.

Referenced for: linear type and ownership terminology.

Chapter 3

Terms, definitions, and symbols

- ¹ For the purposes of this document, the following terms and definitions apply. Terms defined here are used with these precise meanings throughout all normative clauses. Where a term is used in its ordinary English sense rather than as a defined term, it appears without emphasis.

Note

Cross-references of the form (*see §N*) point to the clause where the concept is treated normatively.

3.1 General language terms

abstract syntax tree (AST)

The in-memory tree representation of a Lain program produced by the parser and consumed by semantic analysis.

behavior

External appearance or action of a program as observed by a user or another program. See also: *defined behavior*, *implementation-defined behavior*, *unspecified behavior*, *undefined behavior*.

conforming implementation

An implementation that correctly processes all programs that satisfy the rules of this specification and issues a diagnostic message for every program that violates a **Constraint** (see §4).

conforming program

A program that uses only the syntax and semantics defined in this specification, does not rely on undefined behavior, and does not depend on implementation-defined extensions.

constraint

A requirement imposed on the source text. If a program violates a constraint, the implementation *shall* emit a diagnostic message. Constraints appear in **Constraint** environments throughout this specification.

defined behavior

Behavior that is fully and precisely specified by this document for every conforming program. Contrast with *undefined behavior* and *implementation-defined behavior*.

diagnostic message

A human-readable message issued by the implementation to report a constraint violation. Every diagnostic shall include a source location (file name, line number) and an error code of the form [EXXX] or [VRA] (see Annex B).

ill-formed program

A program that violates one or more of the syntactic or semantic rules of this specification. An ill-formed program is not a conforming program and a conforming implementation shall reject it with a diagnostic.

implementation

The system (compiler, linker, runtime) that translates a Lain source program into an executable. The reference implementation is the *lain* compiler.

implementation-defined behavior

Behavior that is valid but not fully specified by this document; the implementation shall determine the exact behavior and document it.

program

A collection of one or more Lain source files (`.ln`) that together constitute a translation unit or a set of modules.

source file

A UTF-8 encoded text file with the extension `.ln` containing Lain source text.

translation output

The C99 source code emitted by a conforming implementation for a well-formed Lain program.

undefined behavior

Behavior for which this specification imposes no requirements. A program that exhibits undefined behavior is erroneous. In the safe fragment of Lain, no operation produces undefined behavior. Undefined behavior may arise only inside `unsafe` blocks or through direct C interoperability.

unspecified behavior

Behavior for which this specification provides two or more possibilities without prescribing which shall occur. A conforming program shall not depend on unspecified behavior.

well-formed program

A program that satisfies all syntactic and semantic rules of this specification and is therefore accepted by a conforming implementation without diagnostic.

3.2 Type system terms

algebraic data type (ADT)

A composite type that is either a product type (*struct*) or a sum type (*enum*).

constrained type

A type annotated with a value range constraint of the form $T \text{ op bound}$, e.g. `int >= 0` (see §7).

integer rank

A total ordering on integer types that determines the result type of mixed-type arithmetic. The rank system is defined in §7.

integer widening

An implicit conversion from an integer type of lower rank to one of higher rank, permitted in arithmetic and assignment contexts (see §7).

linear type

A type whose values must be used exactly once. Struct types and resource types are linear. Primitive types (`int`, `bool`, etc.) are unrestricted (see §11).

nominal type

A type identified by its declared name rather than its structure.

ownership mode

An annotation on a variable or parameter that specifies the kind of access the holder has: `shared` (read-only), `var` (read-write), or owned (no annotation on a linear type, transferred with `mov`) (see §11).

primitive type

One of the built-in types: `int`, `u8`, `u16`, `u32`, `u64`, `usize`, `i8`, `i16`, `i32`, `i64`, `isize`, `f32`, `f64`, `bool`. Primitive types are unrestricted (non-linear).

structural type equivalence

Two types are structurally equivalent if they have the same structure. Lain uses nominal equivalence for user-defined types; the name is part of the type identity.

unrestricted type

A type whose values may be used any number of times without explicit consumption. All primitive types are unrestricted. Contrast with *linear type*.

3.3 Ownership and memory terms

active borrow

A borrow that is currently in the ACTIVE phase; its subject cannot be moved or mutated while the borrow is active (see § 11).

arena

A memory allocation strategy in which all allocations are made from a contiguous block; the entire block is freed at once. Lain’s compiler allocates all AST nodes from arenas.

borrow

A temporary access to a value without taking ownership. A shared borrow grants read-only access; a mutable borrow grants read-write access (see § 11).

borrow phase

Either RESERVED (the borrow has been registered but is not yet active, during argument evaluation) or ACTIVE (the borrow is in force) (see § 11).

consumed variable

A linear variable whose ownership has been transferred (via `mov`). Reading or transferring a consumed variable is a constraint violation.

linear state

The tracking state of a linear variable: one of *Uninit*, *Unconsumed*, or *Consumed* (see § 11).

move

The transfer of ownership of a value from one variable or storage location to another, performed by the `mov` operator. After a move, the source is in the Consumed state.

non-lexical lifetime (NLL)

A lifetime that ends at the variable’s *last use* rather than at the end of its lexical scope. Lain uses NLL to minimize the duration of borrows (see § 11).

read-write lock invariant

The aliasing invariant: at any program point, for any owned value, there exist either N shared borrows ($N \geq 0$) and no mutable borrow, or exactly one mutable borrow and no shared borrows (see § 11).

resource

A value that represents an external entity (file handle, socket, etc.) whose lifetime must be explicitly managed. Resources are modelled as linear types.

two-phase borrow

A borrow that begins in the RESERVED phase during argument evaluation and is promoted to ACTIVE after the call expression is complete (see § 11).

3.4 Control flow and purity terms

decreasing measure

An expression in a **while decreasing** statement that the implementation shall verify is non-negative when the loop condition holds and strictly decreases on every iteration (see §9).

func

A pure, terminating function. A **func** shall not call a **proc** or contain an unbounded **while** loop (see §12).

purity

The property of a **func**: its result depends only on its inputs and it has no observable side effects on any state outside its own stack frame.

proc

A procedure that may have side effects. A **proc** may call other **procs**, perform I/O, and contain unbounded **while** loops (see §12).

termination

The property that a function always reaches a **return** statement in a finite number of steps. All **funcs** shall be verified to terminate by the implementation (see §12).

3.5 Verification terms

bounds check

A runtime test that verifies an array index is within the valid range $[0, N-1]$. A conforming implementation shall eliminate bounds checks when VRA can statically prove the index is in range.

comptime

Compile-time evaluation. A **comptime** expression or function is evaluated by the compiler at compile time rather than at runtime (see §14).

exhaustiveness

The property of a **case** expression in which every possible value of the scrutinee type is covered by at least one arm (see §15).

in guard

A use of the **in** operator as a loop or conditional guard. An in guard of the form `idx in arr` proves to the VRA that $idx \in [0, arr.len-1]$ in the guarded body (see §8, §13).

interval

A pair $[\ell, u]$ representing the set of integers $\{x \mid \ell \leq x \leq u\}$. VRA represents value ranges as intervals with $-\infty$ and $+\infty$ sentinels.

monomorphization

The process by which a generic function is instantiated for each concrete set of type arguments, producing a separate, non-generic translation for each instantiation (see § 14).

return constraint

A value range constraint on the return value of a function, declared in the function signature as *T op bound* (see § 13).

value range analysis (VRA)

A static analysis that tracks integer value ranges through the program using interval arithmetic, used to eliminate bounds checks and prove constraint satisfaction (see § 13).

3.6 Module and name resolution terms

import

A directive (`import module.path`) that makes the names defined in another module available in the current module (see § 16).

module

The unit of compilation in Lain. Each source file defines one module.

qualified name

A name of the form *module.identifier* that refers to a declaration in a specific module (see § 6).

Universal Function Call Syntax (UFCS)

A syntactic sugar by which `x.f(args)` is desugared to `f(x, args)` before name resolution. UFCS allows methods to be defined as free functions (see § 12).

Chapter 4

Conformance

4.1 Conforming implementation

- ¹ A **conforming implementation** shall:
 1. accept, without diagnostic, every *conforming program*;
 2. reject, with at least one diagnostic message, every program that violates any constraint stated in this specification;
 3. translate accepted programs such that their observable behavior conforms to the semantic rules stated in this specification;
 4. implement all features described as normative in Clauses 5 through 20;
 5. document every behavior described in this specification as *implementation-defined*¹.
- ² A conforming implementation may:
 1. issue additional diagnostic messages (warnings) beyond those required by this specification, provided they do not cause a conforming program to be rejected;
 2. accept programs through implementation-defined extensions, provided those extensions are documented and do not alter the semantics of any conforming program;
 3. omit runtime bounds checks when value range analysis (§ 13) proves the access is within bounds.

Note

The reference implementation of Lain (*lain* compiler, version corresponding to this specification) is a conforming implementation. Where this specification marks a behavior as implementation-defined, the reference implementation's documented behavior takes precedence for programs compiled by that implementation.

4.2 Conforming program

- ¹ A **conforming program** is a program that:
 1. consists only of syntax and semantics defined in this specification;
 2. does not rely on undefined behavior;

¹Implementation-defined behavior: the exact value or behavior is implementation-defined

3. does not rely on unspecified behavior to produce a specific outcome;
 4. does not depend on any implementation-defined extension.
- 2 A **strictly conforming program** additionally:
1. does not use any behavior documented as implementation-defined, even when that behavior is deterministic on a given implementation.

4.3 Diagnostic messages

- 1 A diagnostic message issued for a constraint violation shall include:
1. the source file name;
 2. the line number of the offending construct;
 3. the diagnostic code in the form [E`XXX`] (see Annex B);
 4. a human-readable description of the violation.
- 2 A conforming implementation may issue additional diagnostic information (column number, source context, suggested fix) beyond the minimum required by §4.3.

Note

The reference implementation additionally prints the source line and a caret (^) pointing to the column of the offending token.

4.4 Implementation limits

- 1 A conforming implementation shall support at least the following minimum translation limits. Programs that require more than these limits invoke *implementation-defined*².

Resource	Minimum	Clause
Identifier length (bytes)	255	§5
Block nesting depth	64	§9
Function parameters	127	§10
Array elements	$2^{31} - 1$	§7
Module import depth	32	§16
<code>case</code> arms per match expression	256	§15
Struct fields	256	§7
Enum variants	256	§7
Comptime evaluation depth	256	§14
Scope nesting depth	64	§6

²Implementation-defined behavior: the behavior when a translation limit is exceeded

4.5 Safe and unsafe fragments

- 1 The **safe fragment** of a Lain program consists of all code that does not appear inside an **unsafe** block and does not use raw pointer operations.
- 2 A conforming implementation shall guarantee, for any well-formed program in the safe fragment, that execution produces no:
 - use of a variable after its ownership has been transferred (use-after-move);
 - double transfer of ownership of the same value;
 - access to an uninitialized variable;
 - violation of the read-write lock invariant (§ 3.3);
 - buffer overrun that has been statically proven safe by value range analysis.
- 3 The **unsafe fragment** comprises all code inside **unsafe** blocks. Within the unsafe fragment, the programmer is responsible for upholding the memory safety invariants listed in § 18; the implementation shall not be required to verify them.

Constraint

The constraints of the ownership and linearity system (§ 11) are *not* suspended inside **unsafe** blocks. Only raw pointer operations and the bounds-check requirement are relaxed. An implementation shall continue to enforce [E001]–[E005] inside **unsafe** blocks.

Chapter 5

Lexical conventions

- 1 A Lain source file is translated in two conceptual steps: (1) the source text is decomposed into tokens as described in this clause; (2) the token stream is parsed into an abstract syntax tree according to the grammar. Implementation may combine these steps.

5.1 Source file encoding

- 1 A Lain source file shall be a sequence of bytes encoded in UTF-8 (Unicode Standard, Annex D). The byte order mark (U+FEFF) shall not appear in a source file. If it does, behavior is *implementation-defined*¹.
- 2 Multi-byte UTF-8 sequences are permitted inside string literals and comments but shall not appear in identifiers. Identifiers shall consist solely of ASCII characters (see §5.6).
- 3 Line endings are normalized: a carriage return (U+000D) followed immediately by a line feed (U+000A) is treated as a single line feed. A lone carriage return is treated as a line feed.

5.2 Comments

- 1 A *line comment* begins with `//` and extends to the first following line feed character. The line feed is not part of the comment.
- 2 A *block comment* begins with `/*` and ends with the nearest following `*/`. Block comments may be nested: each opening `/*` inside a block comment opens a new nesting level requiring its own closing `*/`.
- 3 Comments are stripped during lexical analysis and have no effect on the semantics of the enclosing program.

Constraint

An unterminated block comment (one for which no matching `*/` is found before the end of the source file) is *ill-formed* ([E100]).

5.3 Tokens

- 1 The result of lexical analysis is a sequence of tokens. Whitespace and comments serve solely to separate tokens. The grammar production for *token* is given in Annex A (§??).
- 2 Where the token stream is ambiguous, the lexer shall apply the *maximal munch* rule: at each position, the longest sequence of characters that forms a valid token is consumed.

Example

```
1 | ..= // one token: range-inclusive, NOT '.' followed by '.='
```

¹Implementation-defined behavior: whether a leading BOM is silently skipped or causes a diagnostic

```
2 >= // one token: greater-than-or-equal, NOT '>' followed by
   '='
```

- 3 Keywords (§5.5) take priority over identifiers: the character sequence **func** at a point where an identifier is expected is always the keyword **func**.

5.4 Whitespace

- 1 The whitespace characters are: space (U+0020) and horizontal tab (U+0009). They serve only to separate adjacent tokens.
- 2 Line feeds (U+000A), after normalization (§5.1), are significant: they act as statement terminators in contexts where a statement may follow (see §9). Multiple consecutive line feeds are treated as a single statement boundary.

Constraint

A semicolon (;) shall not appear in a Lain source file. If one is present, the implementation shall emit [E100].

Rationale

Newlines as statement terminators eliminate the class of bugs caused by missing or accidental semicolons. They also prevent multi-statement lines, keeping statement boundaries unambiguous.

5.5 Keywords

- 1 The following identifiers are reserved as keywords. They shall not be used as user-defined identifiers. The grammar production is *keyword* (Annex A).

Keyword	Purpose	Clause
and	Logical AND operator	§8
as	Type cast	§8
case	Pattern matching	§15
comptime	Compile-time evaluation	§14
decreasing	Bounded-loop termination measure	§9
defer	Deferred execution	§9
else	Alternative branch	§9
false	Boolean literal	§5.7
for	Range-based iteration	§9
func	Pure function declaration	§12
if	Conditional	§9
import	Module import	§16
in	Range iteration / bounds guard	§8
module	Module declaration	§16
mov	Ownership transfer	§11
mut	Mutable borrow expression	§11
not	Logical NOT	§8

(continued)

Keyword	Purpose	Clause
or	Logical OR operator	§ 8
proc	Procedure declaration	§ 12
ref	Shared borrow expression	§ 11
return	Return statement	§ 9
struct	Struct type declaration	§ 10
enum	Enum type declaration	§ 10
true	Boolean literal	§ 5.7
unsafe	Unsafe block	§ 18
var	Mutable binding / mutable borrow	§ 10
while	Loop statement	§ 9

- 2 The following identifiers are *reserved keywords*: they are recognized by the lexer but have no current semantics. They are reserved for future use.

Reserved keyword	Intended future purpose
end	Block terminator
export	Module visibility control
extern	C interoperability declarations
type	Type alias / type parameter

Constraint

An identifier that is identical to a keyword or reserved keyword shall not be used in any context that requires an identifier. Such use is *ill-formed* ([E100]).

5.6 Identifiers

- 1 An identifier consists of ASCII letters, decimal digits, and underscores, and shall begin with a letter or underscore. The grammar production is *identifier* (Annex A).
- 2 Identifiers are case-sensitive: `foo`, `Foo`, and `F00` are three distinct identifiers.
- 3 A conforming implementation shall support identifiers of at least 255 characters. The behavior when an identifier exceeds this length is *implementation-defined*².

Constraint

An identifier shall not be identical to a keyword (§ 5.5).

Constraint

An identifier shall begin with a letter or underscore, not a digit. A sequence of characters that begins with a digit is an integer literal (§ 5.7), not an identifier.

²Implementation-defined behavior: whether oversized identifiers are truncated or cause a diagnostic

5.7 Literals

5.7.1 Integer literals

- 1 An integer literal denotes a constant integer value. Four notations are supported (grammar: *integer-literal*):

Notation	Prefix	Example
Decimal	(none)	42, 1_000_000
Hexadecimal	0x	0xFF, 0x1A_3B
Binary	0b	0b1010_0101
Octal	0o	0o755

- 2 An underscore (`_`) may appear between any two digits of an integer literal as a visual separator. It has no effect on the value.
- 3 The type of an integer literal is inferred from context (see §7). In the absence of type context, the default type is `int`.

Constraint

An integer literal whose value does not fit in the type determined by context is *ill-formed* ([E100]).

Example

```

1 var a = 255           // int, value 255
2 var b u8 = 255       // u8, value 255 -- OK
3 var c u8 = 256       // [E100]: 256 does not fit in u8
4 var d = 0xFF         // int, value 255
5 var e = 0b1111_0000  // int, value 240
6 var f = 1_000_000    // int, value 1000000

```

5.7.2 Floating-point literals

- 4 A floating-point literal consists of a decimal integer part, a decimal point, and a fractional part. An optional exponent part `eN` or `EN` may follow (grammar: *float-literal*).
- 5 The default type of a floating-point literal is `f64`.

Example

```

1 var x = 3.14         // f64
2 var y f32 = 1.0      // f32
3 var z = 2.5e3        // f64, value 2500.0

```

5.7.3 Boolean literals

- 6 The boolean literals are `true` and `false` (grammar: *bool-literal*). Both have type `bool`.

5.7.4 Character literals

- 7 A character literal is a single byte value enclosed in single quotes. It has type `u8`. Escape sequences (§5.7.6) are permitted.

Example

```

1 var a = 'A'      // u8, value 65
2 var n = '\n'     // u8, value 10
3 var z = '\0'     // u8, value 0

```

Constraint

A character literal shall contain exactly one character or one escape sequence. An empty or multi-character character literal is *ill-formed* ([E100]).

5.7.5 String literals

- 8 A string literal is a sequence of zero or more characters enclosed in double quotes. Escape sequences (§ 5.7.6) are permitted (grammar: *string-literal*).
- 9 The type of a string literal of N visible characters is `Fixed_u8_N+1` (an array of $N+1$ bytes, null-terminated). This type is implicitly coercible to `[u8]` (a byte slice).
- 10 The `.len` property of a string literal value equals N (excluding the null terminator).

Example

```

1 var s = "hello"   // type: Fixed_u8_6 (5 chars + null)
2                  // s.len == 5

```

- 11 String literals are stored in the read-only data segment of the translation output. Modifying a string literal through a pointer is **undefined behavior**³.

5.7.6 Escape sequences

- 12 The following escape sequences are recognized in string and character literals:

Escape	Byte value	Description
<code>\n</code>	0x0A	Line feed (newline)
<code>\t</code>	0x09	Horizontal tab
<code>\r</code>	0x0D	Carriage return
<code>\0</code>	0x00	Null byte
<code>\\</code>	0x5C	Backslash
<code>\"</code>	0x22	Double quote
<code>\'</code>	0x27	Single quote

Constraint

A backslash not followed by one of the characters in the table above is an unrecognized escape sequence and is *ill-formed* ([E100]).

5.8 Operators and punctuators

- 1 The complete set of operator and punctuation tokens is listed below. The grammar productions *operator* and *punctuator* are in Annex A.

³Undefined behavior: the result of writing through a pointer derived from a string literal

5.8.1 Arithmetic operators

Token	Meaning
+	Addition; unary identity
-	Subtraction; unary negation
*	Multiplication
/	Division
%	Modulo (remainder)

5.8.2 Comparison operators

Token	Meaning
==	Equal
!=	Not equal
<	Less than
>	Greater than
<=	Less than or equal
>=	Greater than or equal

5.8.3 Logical operators

Token	Meaning
and	Logical AND (short-circuit)
or	Logical OR (short-circuit)
not	Logical NOT (prefix)
!	Logical NOT (alternate prefix form)

5.8.4 Bitwise operators

Token	Meaning
&	Bitwise AND
 	Bitwise OR
^	Bitwise XOR
«	Left shift
»	Right shift

5.8.5 Assignment operators

Token	Meaning
=	Assignment
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign
%=	Modulo and assign
&=	Bitwise AND and assign
=	Bitwise OR and assign
^=	Bitwise XOR and assign
<<=	Left shift and assign
>>=	Right shift and assign

5.8.6 Other operators and punctuators

Token	Meaning
.	Member access; module path separator
..	Exclusive range (half-open interval)
->	Function return type separator
(	Open parenthesis
)	Close parenthesis
{	Open brace
}	Close brace
[	Open bracket
]	Close bracket
,	List separator
:	Type annotation; match arm separator
@	Attribute / annotation prefix

5.9 Token disambiguation

- Several token sequences have context-dependent meaning that is resolved by the parser, not the lexer. The lexer produces the same token regardless of context:

Symbol	Context 1	Context 2
*	Multiplication (binary)	(future: pointer dereference)
-	Subtraction (binary)	Negation (unary prefix)
.	Member access	Module path separator
&	Borrowed match prefix	Bitwise AND

5.10 Integer arithmetic and overflow

- 1 Integer arithmetic in Lain uses two's complement representation for all signed integer types. Overflow wraps modulo 2^N where N is the bit width of the type.
- 2 Unsigned integer arithmetic is modular (wrapping) by definition.
- 3 A conforming implementation that emits C99 shall compile the translation output with `-fwrapv` or equivalent, ensuring that signed integer overflow is two's complement wrapping and not undefined behavior in the C sense.

Note

This differs from ISO C, where signed integer overflow is undefined behavior. In Lain, signed overflow is defined as wrapping. Programs that rely on wrapping behavior are well-formed.

Chapter 6

Basic concepts

6.1 Declarations and definitions

- 1 A *declaration* introduces a name into a scope and associates it with an entity: a variable, function, procedure, type, or module. Every name shall be declared before it is used, with the exception of names that are resolved by Universal Function Call Syntax (§12).
- 2 A *definition* is a declaration that also allocates storage (for variables) or provides a body (for functions and procedures). In Lain, every declaration at the local level is also a definition.

Constraint

Use of an undeclared identifier is *ill-formed* ([E013]).

Example

```
1 func f() int {  
2     return x    // [E013]: x is not declared  
3 }
```

6.2 Scopes

- 1 A *scope* is a region of program text within which a declared name can be used. Lain has the following scope levels:

Module scope Names declared at the top level of a source file are in module scope. Module-scope names are accessible throughout the module and, by default, accessible from other modules that import this module.

Function/procedure scope Parameter names are in the scope of the function or procedure body.

Block scope Names declared inside a block (`{ }`) are in the scope of that block. They are not accessible outside the enclosing block.

- 2 Scopes are *lexically nested*. A name in an inner scope may *shadow* a name in an outer scope: within the inner scope, the inner name takes precedence.

Note

Shadowing is permitted but the reference implementation may issue a warning. Programs shall not rely on shadowing being silent.

- 3 A conforming implementation shall support at least 64 levels of nested block scope (§4.4).

6.3 Name resolution

- 1 Name resolution maps an identifier in source text to its declaration. The lookup algorithm is:
 1. Search the innermost block scope.
 2. If not found, search progressively wider enclosing block scopes.
 3. If not found, search the function/procedure scope (parameters).
 4. If not found, search the module scope.
 5. If not found, search imported module scopes (in import order).
 6. If not found, attempt UFCS resolution (§ 12).
 7. If still not found, the program is *ill-formed* ([E013]).
- 2 When the same name is visible in multiple scopes, the innermost binding takes precedence (shadowing).

6.4 Storage duration

- 1 Every object has a *storage duration* that determines the lifetime of its storage.

Automatic storage duration Variables declared inside a block or function body. Storage is allocated on entry to the block and deallocated on exit.

Static storage duration Variables declared at module scope. Storage persists for the lifetime of the program. *implementation-defined*¹.

Arena storage duration Objects allocated from an explicit **Arena**. Storage persists until the arena is freed as a whole.

6.5 Objects and values

- 1 An *object* is a region of storage interpreted according to a type. Every declared variable corresponds to an object.
- 2 Every variable has:
 - a *declared type* (§ 7);
 - an *ownership mode*: owned (default for linear types), **shared** (read-only reference), or **var** (mutable reference) (§ 11);
 - a *linear state*: Uninit, Unconsumed, or Consumed (for linear types only) (§ 11).

6.6 Lvalues and rvalues

- 1 An *lvalue* is an expression that designates an object with a persistent storage location: a variable identifier, a field access on an lvalue, or an array index expression on an lvalue.
- 2 An *rvalue* is an expression whose value has no persistent storage location: literals, the result of arithmetic operations, or the result of a function call.
- 3 Assignment requires the left-hand side to be an lvalue.

¹Implementation-defined behavior: whether module-scope variables are zero- initialized before **proc main** is called

Constraint

An assignment whose left-hand side is an rvalue is *ill-formed* ([E100]).

6.7 Alignment

- ¹ Every type has a required alignment. The alignment of a type is *implementation-defined*². Struct layout follows C99 struct layout rules, including padding (§ 7).
- ² Stack-allocated variables are aligned to their type's required alignment. The implementation shall not insert additional padding between local variables unless required by the hardware.

²Implementation-defined behavior: the same as the alignment of the corresponding C99 type

Chapter 7

Types

- 1 Lain is statically typed. Every variable, expression, and parameter has a type determined at compile time. There is no runtime type information (RTTI). The grammar productions for types are in Annex A (*type*).

7.1 Type taxonomy

- 1 The types of Lain are organized as follows:
 - *Primitive types*: integer, floating-point, boolean, void (§ 7.2);
 - *Array types*: fixed-size, compile-time-sized (§ 7.3);
 - *Slice types*: dynamically-sized views into contiguous memory (§ 7.4);
 - *Struct types*: named product types (§ 7.5);
 - *Enum types*: simple enumerations (§ 7.6);
 - *ADT types*: sum types with associated data (§ 7.7);
 - *Pointer types*: raw memory addresses, unsafe only (§ 7.8);
 - *Constrained types*: types with value range annotations (§ 7.9).
- 2 Types are classified as *linear* or *unrestricted*. Linear types must be consumed exactly once (see § 11). Unrestricted types may be used any number of times.
- 3 All primitive types are unrestricted. A struct type is linear if and only if at least one of its fields is of a linear type. This property is transitive.

7.2 Primitive types

7.2.1 Integer types

- 1 The integer types are listed below. Sizes marked (*platform*) are *implementation-defined*¹.

Type	Signedness	Bit width	Range	C99 type
u8	unsigned	8	$[0, 2^8 - 1]$	uint8_t
u16	unsigned	16	$[0, 2^{16} - 1]$	uint16_t
u32	unsigned	32	$[0, 2^{32} - 1]$	uint32_t

¹Implementation-defined behavior: determined by the target platform as in the C99 standard

(continued)

Type	Signedness	Bit width	Range	C99 type
<code>u64</code>	unsigned	64	$[0, 2^{64} - 1]$	<code>uint64_t</code>
<code>usize</code>	unsigned	(platform)	$[0, 2^N - 1]$	<code>size_t</code>
<code>i8</code>	signed	8	$[-2^7, 2^7 - 1]$	<code>int8_t</code>
<code>i16</code>	signed	16	$[-2^{15}, 2^{15} - 1]$	<code>int16_t</code>
<code>i32</code>	signed	32	$[-2^{31}, 2^{31} - 1]$	<code>int32_t</code>
<code>i64</code>	signed	64	$[-2^{63}, 2^{63} - 1]$	<code>int64_t</code>
<code>isize</code>	signed	(platform)	$[-2^{N-1}, 2^{N-1} - 1]$	<code>ptrdiff_t</code>
<code>int</code>	signed	≥ 32	(platform)	<code>int</code>

- 2 `int` is the default integer type; it corresponds to C's `int` and is at least 32 bits wide. Its exact width is *implementation-defined*².
- 3 `usize` and `isize` are the platform-native unsigned and signed integer types whose width equals the pointer width. Their exact width is *implementation-defined*³.

7.2.2 Integer rank and widening

- 4 The integer types are partially ordered by *rank*:

Rank	Types	Bit width
1	<code>u8</code> , <code>i8</code>	8
2	<code>u16</code> , <code>i16</code>	16
3	<code>u32</code> , <code>i32</code> , <code>int</code>	32
4	<code>u64</code> , <code>i64</code>	64
5	<code>usize</code> , <code>isize</code>	platform

- 5 **Implicit widening:** a value of a lower-rank integer type may be used where a higher-rank integer type is expected, without an explicit cast. The result type of a binary arithmetic or bitwise operation is the higher-rank operand type.

Example

```

1 var a u8 = 42
2 var b int = a           // u8 -> int: implicit widening, OK
3 var c = a + 1000        // u8 + int -> int (wider wins)
```

Constraint

An implicit conversion from a higher-rank integer type to a lower-rank integer type (narrowing) is *ill-formed*. Narrowing requires an explicit `as` cast (§8).

Constraint

²Implementation-defined behavior: the width of C's `int` on the target platform

³Implementation-defined behavior: the pointer width on the target platform

An implicit conversion between integer types and floating-point types is *ill-formed*. Such conversions require an explicit `as` cast.

Constraint

An implicit conversion between integer types and `bool` is *ill-formed*. Such conversions require an explicit `as` cast.

7.2.3 Floating-point types

- 6 `f32` and `f64` are 32-bit and 64-bit floating-point types conforming to IEC 60559 (IEEE 754). They correspond to C's `float` and `double` respectively.
- 7 The default type of a floating-point literal is `f64` (§5.7.2).
- 8 Arithmetic on `f32` and `f64` follows IEEE 754 semantics: overflow produces $\pm\infty$; the result of `0.0/0.0` and similar operations is NaN. These are defined behaviors in Lain.

7.2.4 Boolean type

- 9 `bool` has exactly two values: `true` and `false`. `bool` is stored as a single byte; its exact representation is *implementation-defined*⁴.

Constraint

There are no implicit conversions between `bool` and any integer type. `if 1 { }` is *ill-formed*; use `if 1 != 0 { }`.

7.2.5 The void type

- 10 `void` represents the absence of a value. It is used exclusively in pointer types (`*void`) for C interoperability (see §17).

Constraint

A variable declaration with type `void` is *ill-formed*.

7.3 Array types

- 1 An array type `[T; N]` denotes a contiguous sequence of N values of type `T`. N shall be a compile-time constant integer greater than zero (grammar: *array-type*).
- 2 The `.len` property of an array type yields N as a compile-time constant of type `int`.
- 3 Array elements are laid out contiguously in memory in increasing index order, with element 0 at the base address. The stride between elements is *implementation-defined*⁵.

Constraint

An array size $N \leq 0$ is *ill-formed* ([E100]).

Constraint

Every array index expression shall satisfy $0 \leq idx < arr.len$. An access for which value range analysis cannot prove this is a constraint violation ([E014]).

⁴Implementation-defined behavior: whether `true` is 1 or any non-zero value

⁵Implementation-defined behavior: equal to `sizeof(T)` in C99, including any intra-element padding

Example

```

1 var arr = [int; 5]{1, 2, 3, 4, 5}
2 var x = arr[2]           // OK: 2 is statically in [0, 4]
3 // var y = arr[5]       // [E014]: 5 >= arr.len

```

7.4 Slice types

- 1 A slice type `[T]` is a fat pointer to a contiguous sequence of values of type `T` whose length is determined at runtime. Its runtime representation is a pair `(ptr: *T, len: usize)`.
- 2 `str` is a predefined alias for `[u8]`.
- 3 A slice has two properties accessible via dot notation:
 - `.len`: the number of elements, type `usize`;
 - `.data`: a pointer to the first element, type `*T`.
- 4 A string literal of N characters has static type `+1Fixed_u8_N` (a null-terminated fixed-size array of $N+1$ bytes) and is implicitly coercible to `[u8]`.

7.5 Struct types

- 1 A struct type groups named fields into a single named product type. The grammar production is *struct-declaration* (Annex A).
- 2 Field order is the declaration order. The memory layout of a struct is *implementation-defined*⁶.
- 3 A struct type S is linear if at least one field of S has a linear type (transitively).
- 4 **Struct construction**: all fields shall be provided in declaration order. The type of each argument shall be compatible with the declared field type (§ 7.10).

Constraint

A struct constructor expression with a number of arguments different from the number of fields is *ill-formed* ([E100]).

Constraint

Writing to a field of a struct variable requires the variable to be declared as `var` (mutable). Writing to an immutable binding's field is *ill-formed*.

Constraint

Using `==` or `!=` on a struct type is *ill-formed*. Field-by-field comparison must be performed explicitly.

7.6 Enum types

- 1 An enum type is a type with a finite, named set of values. Each variant carries no associated data. The grammar production is *enum-declaration* (Annex A).
- 2 The underlying representation of an enum type is *implementation-defined*⁷.
- 3 Enum values are accessed as `TypeName.VariantName`.

⁶Implementation-defined behavior: equal to the C99 struct layout, including compiler-inserted padding for alignment

⁷Implementation-defined behavior: an integer type large enough to hold all variant tags; the reference implementation uses `int`

Constraint

Using `==` or `!=` to compare enum values is *ill-formed*. Enum values shall be compared using `case` pattern matching (§ 15).

Note

The constraint on `==` for enums reflects that the underlying tag representation is implementation-defined. Pattern matching is always sound.

7.7 Algebraic data types (ADTs)

- 1 An ADT is a sum type in which each variant may carry associated fields. A type declaration is classified as an ADT if at least one variant has fields. The grammar production is *enum-declaration* (Annex A, using the record-variant form).
- 2 ADTs are represented in C as a tagged union: a tag field of implementation-defined integer type, plus a union of variant payload structs.
- 3 ADT values are constructed as `TypeName.VariantName(args)` for variants with fields, or `TypeName.VariantName` for unit variants.
- 4 ADT values are destructured via `case` pattern matching (§ 15).

7.8 Pointer types

- 1 A pointer type `*T` or `const *T` is a raw memory address. Pointer operations require an `unsafe` block (§ 18).
- 2 The ownership modes for pointer types are:

Lain syntax	Mode	C99 type	Linear?
<code>*T</code>	shared (read-only)	<code>const T*</code>	no
<code>var *T</code>	mutable	<code>T*</code>	no
<code>mov *T</code>	owned	<code>T*</code>	yes

Constraint

Dereferencing a pointer or taking the address of a variable shall occur only inside an `unsafe` block. Pointer operations in safe code are *ill-formed* ([E012]).

7.9 Constrained types

- 1 A constrained type is a type annotated with a value range constraint:

$$T \text{ } op \text{ } bound$$

where $op \in \{>=, <=, >, <, ==\}$ and $bound$ is an integer literal.

- 2 Constrained types are used in function return types and parameter types to declare value range invariants that the value range analysis (§ 13) shall enforce.

Example

```
1 func nonzero_divisor() int >= 1 {
2     return 42 // VRA verifies: 42 >= 1 holds
```

```
3 }  
}
```

- 3 The constraint is part of the function signature, not the type system's nominal identity. Two constrained types with the same base type are assignment-compatible.

7.10 Type compatibility and equivalence

- 1 Lain uses **nominal type equivalence**: two types are the same if and only if they have the same declared name.
- 2 Two types T_1 and T_2 are *compatible* for an expression context if:
- $T_1 = T_2$ (same nominal type), or
 - T_1 is an integer type of lower rank than T_2 (implicit widening; § 7.2.2).

Constraint

An expression of type T_1 used in a context requiring type T_2 , where T_1 and T_2 are not compatible, is *ill-formed*.

- 3 The operators `==` and `!=` are defined for:
- primitive numeric types;
 - `bool`;
 - pointer types.

They are not defined for struct types, array types, or enum types.

Chapter 8

Expressions

- 1 An expression is a syntactic construct that evaluates to a value. Every expression has a type, determined statically at compile time. The grammar production for *expression* is in Annex A.
- 2 Expressions are evaluated strictly left-to-right: for any binary operator *a op b* or function call *f(a, b)*, the left operand (or earlier argument) is fully evaluated before the right operand (or later argument).

8.1 Operator precedence and associativity

- 1 The following table lists all binary operators in order of decreasing precedence. All operators listed on the same row have equal precedence.

Prec.	Operators	Assoc.	Result type
1 (high)	() [] .	left	(context-dependent)
2	unary - ! not mov ref mut	right	(operand type)
3	as	left	target type
4	* / %	left	wider operand type
5	+ -	left	wider operand type
6	<< >>	left	left operand type
7	&	left	wider operand type
8	^	left	wider operand type
9		left	wider operand type
10	< > <= >= in	left	bool
11	== !=	left	bool
12	and	left	bool
13 (low)	or	left	bool

- 2 Parentheses override precedence.

8.2 Primary expressions

- 1 A primary expression is one of: an identifier, a literal, a parenthesized expression, a block expression, an **if** expression, a **case** expression, a constructor expression, or a **comptime** expression.
- 2 **Identifier expression:** evaluates to the current value of the named variable.

Constraint

The identifier shall be declared in scope ([E013]).

Constraint

The identifier shall have been initialized before use ([E005]).

8.3 Arithmetic operators

- 1 The binary arithmetic operators are + (addition), - (subtraction), * (multiplication), / (division), and % (remainder). The result type is the wider of the two operand types (§7.2.2).
- 2 Integer division truncates toward zero. `f32/f64` division follows IEEE 754.

Constraint

Integer division or remainder where the divisor is statically provable to be zero (via value range analysis, §13) is *ill-formed* ([E015]).

Constraint

% shall not be applied to floating-point operands.

- 3 The unary - operator negates its operand. For unsigned integer types the result wraps modulo 2^N .

8.4 Comparison operators

- 1 The comparison operators ==, !=, <, >, <=, >= compare two values and yield `bool`.

Constraint

Operands of a comparison operator shall have compatible types (§7.10).

Constraint

== and != shall not be applied to struct types, array types, or enum types (§7.6).

8.5 Logical operators

- 1 `and` and `or` are short-circuit operators: the right operand is not evaluated if the result is determined by the left operand alone. `not` and ! are prefix unary negation. All yield `bool`.

Constraint

Operands of `and`, `or`, `not`, and ! shall have type `bool`.

8.6 Bitwise operators

- 1 The bitwise operators &, |, ^, <<, >> operate on integer operands. The result type is the wider operand type for &, |, ^; for shift operators, the result type is the left operand type.

Constraint

Operands of bitwise operators shall be integer types.

Constraint

The shift amount (right operand of `<<` and `>>`) shall satisfy $0 \leq \text{shift} < N$ where N is the bit width of the left operand. A shift amount outside this range is *ill-formed* ([E100]).

8.7 The `in` operator

- 1 The expression `idx in arr` evaluates to `true` if and only if $0 \leq \text{idx} < \text{arr.len}$. The operand `arr` shall be an array or slice type; `idx` shall be an integer type.
- 2 When `idx in arr` appears as the condition of a `while` or `if` statement (or as the leftmost operand of an `and`-chain condition), it establishes an **in guard**: inside the body of that statement, the implementation shall accept `arr[idx]` without requiring further bounds verification.
- 3 In-guards propagate through `and`-chains: if the left operand of `and` establishes an in-guard, that guard is active when evaluating the right operand.

Example

```

1 // In-guard: arr[i] is safe without explicit VRA
2 while i in arr {
3     var x = arr[i]    // OK: in-guarded
4     i = i + 1
5 }
6
7 // And-chain propagation:
8 while l.pos in l.src and (l.src[l.pos] as int) != ',' {
9     l.pos = l.pos + 1
10 }
```

Constraint

An in-guard of the form `idx in arr` covers only `arr[idx]` exactly. Accesses `arr[idx + k]` for any $k \neq 0$ are not covered by this guard and require separate verification or an `unsafe` block.

8.8 Cast expression (`as`)

- 1 The expression `expr as T` explicitly converts the value of `expr` to type `T`. The following casts are permitted:

From	To	Semantics
integer	integer	Truncation (narrowing) or extension (widening)
integer	float	Nearest representable value (IEEE 754)
float	integer	Truncation toward zero
float	float	Precision change
pointer	pointer	Reinterpretation (<code>unsafe</code> required)
integer	pointer	Reinterpretation (<code>unsafe</code> required)
pointer	integer	Reinterpretation (<code>unsafe</code> required)

Constraint

Casts between pointer types, between a pointer and an integer, or between any type and a pointer shall occur only inside an `unsafe` block. Such a cast outside `unsafe` is *ill-formed* ([E012]).

Constraint

A cast between non-numeric, non-pointer types (e.g., struct to integer) is *ill-formed*.

8.9 Move expression (`mov`)

- 1 `mov` `expr` transfers ownership of the value produced by `expr`. The source is placed in the Consumed state; any subsequent use of the source variable is *ill-formed* ([E001]). See § 11.

8.10 Borrow expressions (`ref`, `mut`)

- 1 `ref` `expr` creates a shared (read-only) borrow of `expr`. `mut` `expr` creates a mutable borrow of `expr`. Both expressions are evaluated in argument contexts to pass values without transferring ownership. See § 11.

8.11 Field access

- 1 The expression `expr.field` accesses the named field of a struct or ADT value, or a property of an array or slice (`.len`, `.data`).

Constraint

The field name shall exist on the type of `expr`.

Constraint

Writing to a field (`expr.field = v`) requires `expr` to be a mutable lvalue.

8.12 Index expression

- 1 The expression `arr[idx]` accesses the element of array or slice `arr` at integer index `idx`.

Constraint

The value range analysis shall prove $0 \leq idx < arr.len$ at every index expression. An index expression for which this cannot be proved is *ill-formed* ([E014]). An `in` guard (§ 8.7) or an `unsafe` block suppresses this requirement.

8.13 Function call

- 1 A function call expression `f(args)` evaluates the arguments and transfers control to the callee. Arguments are evaluated left-to-right.
- 2 Argument ownership modes shall be given explicitly at the call site:

Syntax	Meaning
<code>f(x)</code>	shared borrow (default; parameter mode must be <code>shared</code>)
<code>f(var x)</code>	mutable borrow (parameter mode must be <code>var</code>)

Syntax	Meaning
<code>f(mov x)</code>	ownership transfer (parameter mode must be owned/ mov)

Constraint

The mode annotation at the call site shall match the declared parameter mode. A mismatch is *ill-formed*.

Constraint

The number of arguments shall equal the number of parameters (unless the callee is a variadic C function declared with `...`).

- 3 **Two-phase borrows:** during argument evaluation, borrows of the receiver of a method call are placed in the RESERVED phase and promoted to ACTIVE after all arguments are evaluated (see §11).

8.14 Block expression

- 1 A block expression `{ stmts... expr }` evaluates each statement in order, then evaluates the final expression, whose value is the value of the block. If the final expression is absent, the block has no value.

8.15 If expression

- 1 An **if** expression of the form `if cond { e1 } else { e2 }` evaluates *cond*; if true, evaluates *e*₁; otherwise, evaluates *e*₂.

Constraint

The types of *e*₁ and *e*₂ shall be compatible. If they are not, the expression is *ill-formed*.

Constraint

cond shall have type `bool`.

8.16 Match expression (case)

- 1 A **case** expression (*match-expression*) selects one of several alternatives based on pattern matching. All arms shall produce values of compatible types; the type of the **case** expression is that common type. See §15 for full exhaustiveness rules.

8.17 Constructor expression

- 1 A constructor expression builds a value of a struct or ADT type by supplying all field values. See §7.5 and §7.7 for constructor rules.

8.18 Compound assignment

- 1 Compound assignment `x op= e` is syntactic sugar for `x = x op e`. The left-hand side shall be a mutable lvalue. All compound forms of the arithmetic and bitwise operators are supported (see §5.8).

Chapter 9

Statements

- 1 A statement is a unit of execution. Statements are terminated by line feeds (§ 5.4). The grammar production for *statement* is in Annex A.

9.1 Block statement

- 1 A block statement `{ stmts... }` creates a new scope (§ 6.2) and executes each contained statement in order. Names declared inside a block are not accessible outside it.

9.2 Variable declaration statement

- 1 `var name [: Type] = expr` declares and initializes a local variable. The type may be omitted if it can be inferred from the initializer.

Constraint

A variable declaration shall include either an explicit type annotation or an initializer expression. A declaration with neither is *ill-formed* ([E100]).

Constraint

When both a type annotation and an initializer are present, the type of the initializer shall be compatible with the declared type (§ 7.10).

9.3 Assignment statement

- 1 An assignment statement `lvalue = expr` stores the value of *expr* into the storage designated by *lvalue*.

Constraint

The left-hand side shall be a mutable lvalue (§ 6.6). Assigning to an immutable binding is *ill-formed* ([E009]).

Constraint

Overwriting a linear variable whose current value has not been consumed would leak that value. Such an assignment is *ill-formed* ([E003]).

Note

Assigning to a linear variable after its previous value has been consumed (e.g., loop reassign-and-consume pattern) is permitted. The implementation resets the variable's linear state to Unconsumed after a legal overwrite.

Example

```

1 import std.fs
2
3 proc main() int {
4     var f = open_file("data.txt", "r")
5     close_file(mov f)           // f is now Consumed
6     f = open_file("data.txt", "r") // OK: overwriting Consumed
7     state
8     close_file(mov f)
9     return 0
}
```

9.4 Expression statement

- 1 An expression followed by a statement terminator is an expression statement. The value of the expression is discarded.

9.5 If statement

- 1 **if** *cond* { *body* } [**else if** *cond* { *body* }]... [**else** { *body* }] evaluates *cond*; if true, executes the then-body; otherwise, tests the **else if** clauses in order; if none matches, executes the **else** body (if present).

Constraint

cond shall have type **bool**.

- 2 **Range refinement**: inside the then-branch of **if** $x < N$ { }, the value range analysis (§13) knows that $x \in (-\infty, N - 1]$; inside the else-branch, $x \in [N, +\infty)$.
- 3 **In-guard**: when *cond* contains or is an **in** expression (§8.7), the in-guard is active inside the then-branch only.
- 4 **Branch consistency**: if one branch consumes a linear variable, all branches shall consume it. If branches disagree on the linear state of a variable from the enclosing scope, the program is *ill-formed* ([E003]).

9.6 While statement

- 1 **while** *cond* { *body* } repeatedly evaluates *cond* and, if true, executes *body*.

Constraint

An unbounded **while** loop (one without a **decreasing** measure) shall not appear inside a **func**. Such use is *ill-formed* ([E011]).

Constraint

cond shall have type **bool**.

- 2 **In-guard**: when *cond* contains or is an **in** expression, the in-guard is active inside the loop body for each iteration.

9.6.1 Bounded while statement

- 3 **while** *cond* **decreasing** *M* { *body* } is a bounded loop. In addition to the while semantics above, the implementation shall statically verify:
1. **Non-negativity** ([E080]): the implementation shall prove that $M \geq 0$ whenever *cond* is true.
 2. **Variable dependency** ([E081]): *M* shall reference at least one variable that is assigned in *body*. An *M* that references no such variable would never decrease.
 3. **Strict decrease** ([E082]): the implementation shall prove that every execution of *body* strictly decreases the value of *M*.
- 4 A bounded **while** loop is permitted inside a **func** because the implementation has statically verified termination.
- 5 The following patterns are accepted by the standard verification algorithm:

Condition	Measure	Why
$i < n$	$n - i$	i increases $\Rightarrow$ measure decreases
$n > 0$	n	n decreases directly
$a < b$	$b - a$	either a increases or b decreases
i in <i>arr</i>	$\text{arr.len} - i$	in implies $i < \text{arr.len}$

Example

```

1 func count(n int) int {
2     var i = 0
3     while i < n decreasing n - i {
4         i = i + 1
5     }
6     return i
7 }
```

Rationale

The **decreasing** keyword bridges the gap between pure functions (which must terminate) and the common pattern of finite-input state machines (lexers, parsers) that are provably terminating. The programmer states the termination argument; the compiler verifies it.

9.7 Return statement

- 1 **return** [*expr*] terminates execution of the current function or procedure and transfers the value of *expr* (if present) to the caller.

Constraint

At the point of a **return**, all linear variables in scope shall have been consumed. A live (Unconsumed) linear variable at **return** is *ill-formed* ([E003]).

Constraint

`return var x` shall not return a mutable reference to a local variable. Such a return would create a dangling reference and is *ill-formed* ([E010]).

9.8 Defer statement

- 1 `defer { body }` schedules *body* for execution immediately before the enclosing scope exits, on all exit paths (`return`, `break`, `continue`, or fall-through). Multiple `defer` statements execute in LIFO order.
- 2 **Interaction with linear types:** if a linear variable is consumed inside a `defer` body, the variable is marked as defer-consumed. It is treated as unconsumed for the purposes of the main body (it may be used before the scope exit) and as consumed at scope exit.

Constraint

A `defer` body shall not contain `return`, `break`, or `continue` statements that would transfer control out of the deferred body.

Example

```

1 import std.fs
2
3 proc process() int {
4     var f = open_file("data.txt", "r")
5     defer { close_file(mov f) }    // f consumed at scope exit
6     // ... use f ...
7     return 0
8 }
```

9.9 Unsafe block

- 1 `unsafe { body }` is an unsafe block (see §18). It creates a scope in which certain static checks are relaxed.

9.10 Comptime if statement

- 1 `comptime if cond { then } [else { else }]` evaluates *cond* at compile time. Only the taken branch is type-checked and emitted; the other branch is discarded. *cond* shall be a compile-time constant expression (§14).

Note

`comptime if` is the Lain equivalent of C preprocessor `#if`: it enables platform-specific or type-specific code selection without dead-code warnings.

9.11 For statement

- 1 `for i in start..end { body }` iterates with *i* ranging from *start* (inclusive) to *end* (exclusive). The loop variable *i* is immutable; its value range is statically known to be $[start, end - 1]$ throughout the body, enabling safe array indexing without runtime bounds checks.
- 2 `for` loops are permitted in both `func` and `proc`.

Constraint

start and *end* shall have integer types.

Chapter 10

Declarations

Chapter 11

Ownership

Chapter 12

Functions

Chapter 13

Vra

Chapter 14

Generics

Chapter 15

Match

Chapter 16

Modules

Chapter 17

Interop

Chapter 18

Unsafe

Chapter 19

Memory

Chapter 20

Stdlib

Grammar

Diagnostics

Rationale

Comparison

Index

- algebraic data type, 7, 31
- alignment, 25
- arena, 8
- arithmetic operator, 34
- array type, 29
- as operator, 35
- assignment statement, 39

- basic concepts, 23
- bitwise operator, 34
- block expression, 37
- block statement, 39
- boolean literal, 18
- boolean type, 29
- borrow, 8
 - active, 8
 - phase, 8
- borrow expression, 36
- bounded while loop, 41
- bounds check, 9

- case expression, 37
- cast expression, 35
- character literal, 18
- comment, 15
- comparison operator, 34
- compilation model, 2
- compound assignment, 37
- comptime, 9
- comptime if, 42
- conformance, 11
- conforming implementation, 11
- conforming program, 11
- constrained type, 7, 31
- constructor expression, 37

- declaration, 23
- decreasing measure, 9, 41
- defer statement, 42
- definition, 23
- diagnostic message, 6
 - requirements, 12

- enum type, 30
- escape sequence, 19
- exhaustiveness, 9
- expression, 33
- expression statement, 40

- field access, 36
- floating-point literal, 18
- floating-point type, 29
- for statement, 42
- func, 9
- function call, 36

- identifier, 17
- if expression, 37
- if statement, 40
- implementation limits, 12
- implementation-defined behavior, 6
- import, 10
- in guard, 9, 35
- in operator, 35
- index expression, 36
- integer literal, 18
- integer overflow, 22
- integer rank, 7, 28
- integer type, 27
- integer widening, 7, 28
- interval, 9

- keyword, 16

- lexical conventions, 15
- linear state, 8
- linear type, 7
- literal, 18
- logical operator, 34
- lvalue, 24

- module, 10
- monomorphization, 10
- move, 8
- move expression, 36

- name resolution, 24

- non-lexical lifetime, 8
- normative references, 3
- object, 24
- operator, 19
- operator precedence, 33
- ownership mode, 7
- pointer type, 31
- primary expression, 33
- primitive type, 7, 27
- proc, 9
- purity, 9
- qualified name, 10
- read-write lock invariant, 8
- resource, 8
- return constraint, 10
- return statement, 41
- rvalue, 24
- safe fragment, 13
- scope, 23
- scope (of document), 1
- slice type, 30
- source file, 6
 - encoding, 15
- statement, 39
- storage duration, 24
- string literal, 19
- struct type, 30
- termination, 9
- terms and definitions, 5
- token, 15
- token disambiguation, 21
- two-phase borrow, 8
- type, 27
 - taxonomy, 27
- type compatibility, 32
- type equivalence, 32
- UFCS, 10
- undefined behavior, 6
- Universal Function Call Syntax, 10
- unrestricted type, 7
- unsafe block, 42
- unsafe fragment, 13
- unspecified behavior, 6
- value range analysis (VRA), 10
- variable
 - consumed, 8
- variable declaration, 39
- void type, 29
- while statement, 40
- whitespace, 16